

APIOjek

APIOjek adalah layanan penyewaan sepeda yang baru saja dibuka di Kota APIO. Sejumlah titik penyewaan tersebar di berbagai penjuru kota, yang memungkinkan para penggunanya meminjam atau mengembalikan sepeda pada titik penyewaan mana pun. Karena praktis dan murah, layanan ini dengan cepat menjadi moda transportasi utama di Kota APIO. Akan tetapi, APIOjek juga menghadapi masalah utama yang dialami layanan penyewaan sepeda pada umumnya: ketidakseimbangan tingkat penyewaan dan pengembalian di antara titik-titik yang ada. Akibatnya, di beberapa titik hanya tersedia sangat sedikit sepeda sehingga tidak cukup untuk memenuhi kebutuhan warga sekitar, sementara titik-titik lainnya kelebihan sepeda yang melampaui kebutuhan warga sekitarnya.

Untuk mengatasi masalah ini, APIOjek berencana mengirim sebuah truk penyeimbang setiap malamnya untuk memindahkan sepeda, sehingga setiap titik kembali memiliki banyaknya sepeda yang sesuai. Terdapat N titik di Kota APIO yang dinomori $0, 1, \dots, N - 1$. Dari pengamatan sebelumnya, APIOjek mendapati bahwa setiap petang, banyaknya sepeda di titik i selalu tetap, yakni $A[i]$, dan tidak terjadi peminjaman atau pengembalian sepeda sepanjang malam. APIOjek ingin agar ada tepat $B[i]$ sepeda di titik i pada pagi hari.

Pada N titik ini, terdapat $N - 1$ jalan khusus untuk truk penyeimbang. Masing-masing jalan menghubungkan dua titik yang berbeda, dan truk hanya dapat berpindah menggunakan jalan-jalan ini. Setiap jalan memiliki panjang satu satuan. Jaringan titik-titik terhubung, sehingga dari titik mana pun, truk selalu dapat mencapai titik lainnya. Setiap malam, APIOjek harus mengirim sebuah truk penyeimbang untuk memastikan titik i memiliki tepat $B[i]$ sepeda. Truk ini boleh memulai dan mengakhiri rutenya di titik mana pun.

Truk tersebut berada dalam keadaan kosong pada awal proses penyeimbangan. Ketika truk tersebut berada di suatu titik, sopir truk dapat mengangkat berapa pun sepeda yang ada pada titik tersebut ke dalam truk, atau menurunkan berapa pun sepeda dari truk ke titik itu. Truk dan setiap titik memiliki daya tampung sepeda yang tak hingga, namun banyaknya sepeda di suatu tempat tidak boleh kurang dari nol. APIOjek ingin menentukan jarak terpendek yang mungkin ditempuh untuk menyelesaikan penyeimbangan serta strategi yang harus diambil.

Detail Implementasi

Anda harus mengimplementasikan prosedur berikut.

```
std::pair<std::vector<int>, std::vector<long long>>
find_rebalancing_strategy(int N,
                           std::vector<int> A,
                           std::vector<int> B,
                           std::vector<int> U,
                           std::vector<int> V)
```

- A : array sepanjang N , dengan $A[i]$ menyatakan banyaknya sepeda pada titik i setiap petang.
- B : array sepanjang N , dengan $B[i]$ menyatakan target banyaknya sepeda pada titik i setiap pagi.
- U, V : array sepanjang $N - 1$ yang menyatakan jaringan jalan. Untuk setiap $0 \leq i < N - 1$, jalan i menghubungkan titik $U[i]$ dan titik $V[i]$.
- Prosedur ini dipanggil **paling banyak 150 000 kali** untuk setiap kasus uji.

Prosedur ini harus mengembalikan sepasang array (X, Y) dengan panjang yang sama, yakni $k + 1$, yang menyatakan strategi penyeimbangan:

- X : urutan titik-titik yang dikunjungi secara berurut waktu.
- Y : perubahan neto banyaknya sepeda pada setiap titik yang dikunjungi. Untuk setiap $0 \leq j \leq k$:
 - Jika $Y[j] \geq 0$, sopir truk menurunkan $Y[j]$ sepeda ke titik $X[j]$ dari truk, sehingga banyaknya sepeda pada titik tersebut bertambah sebanyak $Y[j]$.
 - Jika $Y[j] < 0$, sopir truk mengangkut $-Y[j]$ sepeda dari titik $X[j]$ ke truk, sehingga banyaknya sepeda pada titik tersebut berkurang sebanyak $-Y[j]$.

Strategi penyeimbangan (X, Y) dengan jarak k yang **valid** harus memenuhi kondisi-kondisi berikut:

- Untuk setiap $0 \leq j \leq k$, $0 \leq X[j] < N$.
- Untuk setiap $0 \leq j < k$, titik $X[j]$ dan $X[j + 1]$ terhubung langsung oleh suatu jalan.
- Untuk setiap $0 \leq j \leq k$, $\sum_{t=0}^j Y[t] \leq 0$.
- Untuk setiap titik $0 \leq i < N$ dan setiap tahap $0 \leq j \leq k$, misalkan $S_{i,j}$ adalah jumlah $Y[t]$ atas semua tahap t yang memenuhi $0 \leq t \leq j$ dan $X[t] = i$.
 - Banyaknya sepeda pada titik i tidak pernah kurang dari nol: $A[i] + S_{i,j} \geq 0$.
 - Pada akhir strategi, setiap titik i harus memiliki tepat $B[i]$ sepeda: $A[i] + S_{i,k} = B[i]$.

Batasan

- $2 \leq N \leq 300\,000$

- Jumlah nilai N dari semua pemanggilan `find_rebalancing_strategy` tidak melebihi 300 000 pada setiap kasus uji.
- $0 \leq U[i], V[i] < N$ dan $U[i] \neq V[i]$ untuk setiap i sehingga $0 \leq i < N$.
- $0 \leq A[i], B[i] \leq 10^9$ untuk setiap i sehingga $0 \leq i < N$.
- Setiap titik dapat mencapai semua titik lainnya.
- $\sum_{i=0}^{N-1} A[i] = \sum_{i=0}^{N-1} B[i]$.
- Terdapat setidaknya satu nilai i sehingga $0 \leq i < N$ dan $A[i] \neq B[i]$.

Subsoal dan Penilaian

Di sini, T merupakan banyaknya pemanggilan `find_rebalancing_strategy`, dan $\sum N$ merupakan jumlah nilai N dari semua pemanggilan `find_rebalancing_strategy`.

Subsoal	Nilai	Batasan Tambahan
1	4	Titik-titik dan jalan-jalan memenuhi sifat P (dijelaskan di bawah). Terdapat tepat satu nilai i sehingga $0 \leq i < N$ dan $A[i] > 0$.
2	11	$T \leq 10$. $N \leq 7$. Titik-titik dan jalan-jalan memenuhi sifat P.
3	16	Titik-titik dan jalan-jalan memenuhi sifat P.
4	9	Terdapat tepat satu nilai i sehingga $0 \leq i < N$ dan $A[i] > 0$.
5	24	$\sum N \leq 500$.
6	15	$\sum N \leq 5000$.
7	21	Tidak ada batasan tambahan.

Sifat P: Untuk setiap $0 \leq i < N - 1$, $U[i] = i$ dan $V[i] = i + 1$. Untuk setiap $0 \leq i < N$, $A[i] \neq B[i]$.

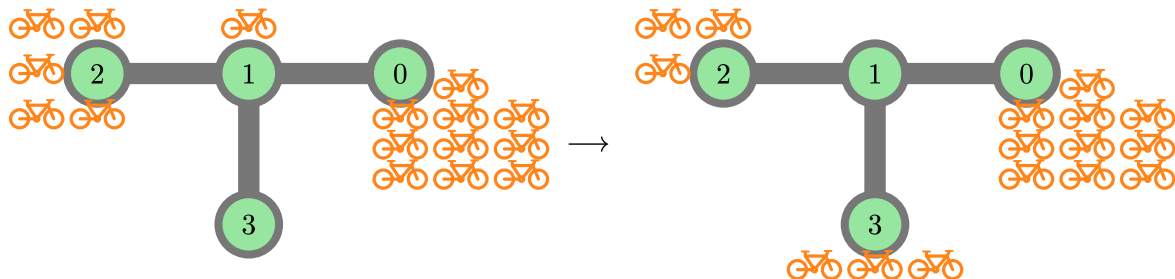
Jika *array* kembalian X dan Y memiliki panjang tepat $k^* + 1$, dengan k^* merupakan jarak terpendek yang mungkin, namun (X, Y) bukan strategi yang valid, Anda akan mendapatkan 50% poin.

Contoh

Contoh 1

Perhatikan pemanggilan berikut:

```
find_rebalancing_strategy(4,
                        [10, 1, 5, 0],
                        [10, 0, 3, 3],
                        [0, 1, 1],
                        [1, 2, 3])
```



Terdapat $N = 4$ titik di Kota APIO. Pada awalnya, titik-titik tersebut memiliki $A = [10, 1, 5, 0]$ sepeda, dan kita ingin agar terdapat $B = [10, 0, 3, 3]$ sepeda pada masing-masing titik.

Misalkan truk penyeimbang mulai dari titik 2 dan mengikuti tahap-tahap berikut:

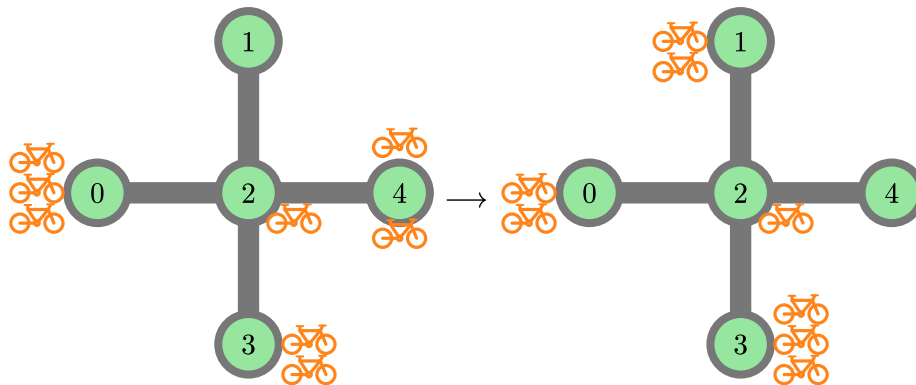
- Pada titik 2, muat 2 sepeda ke truk. Titik 2 kini memiliki 3 sepeda dan truk menampung 2 sepeda.
- Pindah ke titik 1 dan muat 1 sepeda ke truk. Titik 1 sekarang memiliki 0 sepeda dan truk menampung 3 sepeda.
- Pindah ke titik 3 dan turunkan 3 sepeda. Titik 3 sekarang memiliki 3 sepeda dan truk menampung 0 sepeda.

Jarak tempuh keseluruhan adalah 2. *Array* kembalian yang bersesuaian dengan strategi ini adalah $X = [2, 1, 3]$ dan $Y = [-2, -1, 3]$. Dapat dibuktikan bahwa strategi ini menggunakan jarak terpendek yang mungkin. Jadi, prosedur ini dapat mengembalikan $([2, 1, 3], [-2, -1, 3])$.

Contoh 2

Perhatikan pemanggilan berikut:

```
find_rebalancing_strategy(5,
                        [3, 0, 1, 2, 2],
                        [2, 2, 1, 3, 0],
                        [2, 2, 2, 2],
                        [0, 4, 3, 1])
```



Terdapat $N = 5$ titik di Kota APIO. Pada awalnya, titik-titik tersebut memiliki $A = [3, 0, 1, 2, 2]$ sepeda, dan kita ingin agar terdapat $B = [2, 2, 1, 3, 0]$ sepeda pada masing-masing titik.

Misalkan truk penyeimbang mulai dari titik 0 dan mengikuti tahap-tahap berikut:

- Pada titik 0, muat 1 sepeda.
- Pindah ke titik 2 dan muat 1 sepeda.
- Pindah ke titik 1 dan turunkan 2 sepeda.
- Pindah ke titik 2.
- Pindah ke titik 4 dan muat 2 sepeda.
- Pindah ke titik 2 dan turunkan 1 sepeda.
- Pindah ke titik 3 dan turunkan 1 sepeda.

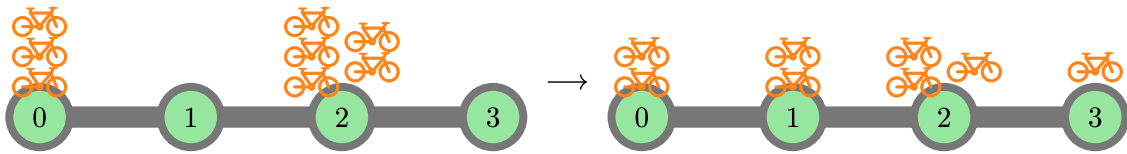
Jarak tempuh keseluruhan adalah 6. *Array* kembalian yang bersesuaian dengan strategi ini adalah $X = [0, 2, 1, 2, 4, 2, 3]$ dan $Y = [-1, -1, 2, 0, -2, 1, 1]$. Dapat dibuktikan bahwa strategi ini menggunakan jarak terpendek yang mungkin. Jadi, prosedur ini dapat mengembalikan $([0, 2, 1, 2, 4, 2, 3], [-1, -1, 2, 0, -2, 1, 1])$.

Strategi valid lainnya dengan jarak 6, seperti $X = [0, 2, 3, 2, 4, 2, 1]$ dan $Y = [-1, 0, 1, 0, -2, 0, 2]$, juga dianggap benar. Mengembalikan *array* X, Y dengan panjang $7 = 6 + 1$ yang tidak berisi strategi yang valid, seperti $X = Y = [0, 0, 0, 0, 0, 0, 0]$, akan mendapatkan 50% nilai.

Contoh 3

Perhatikan pemanggilan berikut:

```
find_rebalancing_strategy(4,
                        [3, 0, 5, 0],
                        [2, 2, 3, 1],
                        [0, 1, 2],
                        [1, 2, 3])
```



Salah satu solusi optimal adalah $X = [2, 1, 0, 1, 2, 3]$ dan $Y = [-1, 1, -1, 1, -1, 1]$. Prosedur ini dapat mengembalikan $([2, 1, 0, 1, 2, 3], [-1, 1, -1, 1, -1, 1])$. Strategi valid lainnya, seperti $X = [2, 1, 0, 1, 2, 3]$ dan $Y = [-2, 2, -1, 0, 0, 1]$, juga dianggap benar.

Contoh ini memenuhi batasan subsoal 2 dan 3.

Contoh Grader

Baris pertama masukan harus berisi sebuah bilangan bulat T yang menyatakan banyaknya skenario. Masukan diikuti oleh deskripsi dari T skenario tersebut, masing-masing dalam format di bawah ini.

Format masukan:

```
N
A[0] A[1] ... A[N-1]
B[0] B[1] ... B[N-1]
U[0] V[0]
U[1] V[1]
...
U[N-2] V[N-2]
```

Format keluaran:

```
k
X[0] X[1] ... X[k]
Y[0] Y[1] ... Y[k]
```